

□ Exercice 1

Une réponse brève d'une ligne est attendue dans le cadre qui suit immédiatement la question, on ne *demande pas* de justification.

1. En Python, quelle est la valeur de la variable `a` après exécution de l'instruction `a = (16 // 5)**3 - 2`

25

2. Si la variable `x` vaut 42, quelle est la valeur de `x % 7 == 0 and (x < 30 or x % 2 == 0)` ?

True

3. Si `lst` vaut `[3, 7, 12, 18, 25]`, que vaut `lst[-1] - lst[1] + len(lst)` ?

23

4. Pour quelle(s) valeurs de la variable `i` sera effectuée la boucle `for i in range(20, 5, -4)`

20, 16, 12, 8

5. Si `s` est la chaîne de caractères "Python" quel est l'affichage produit par `print(s[:2] * 2 + s[-1])`

"PyPyn"

6. Écrire en une seule instruction Python l'affectation permettant d'échanger les valeurs de deux variables `a` et `b` sans variable intermédiaire.

`a, b = b, a`

7. Expliquer l'origine de l'erreur `TypeError: 'tuple' object does not support item assignment` en Python.

Cela signifie qu'on a tenté de modifier un élément d'un tuple, or le type `tuple` n'est pas mutable (il est immuable).

8. Écrire l'instruction conditionnelle permettant de tester si un entier `n` est compris entre 10 et 50 (inclus) et est divisible par 3.

`if 10 <= n <= 50 and n % 3 == 0:`

9. Écrire une instruction permettant de créer *par compréhension* la liste des carrés des entiers de 1 à 10 inclus : `[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]`

`l = [k**2 for k in range(1, 11)]`

10. Quel sera le contenu de la liste `a` après exécution des instructions suivantes :

```
1 a = [10, 20, 30]
2 b = a
3 b[1] = 99
4 a.append(40)
```

`[10, 99, 30, 40]`

□ Exercice 2 : QCM

Dans cet exercice, une question peut avoir *zéro, une ou plusieurs bonnes réponses*. Pour chaque question, cocher les cases correspondantes aux bonnes réponses.

- Que peut-on dire du résultat de l'expression `15 // 4 + 7 / 2` ?
☐ C'est un `int` ☒ C'est un `float` ☐ Sa valeur est 6 ☒ Sa valeur est 6.5
- Quelles sont les propositions exactes concernant les fonctions en Python ?
☒ Une fonction sans instruction `return` explicite renvoie la valeur `None`
☐ Les arguments d'une fonction doivent obligatoirement être du même type
☒ Une fonction peut renvoyer plusieurs valeurs séparées par des virgules
☒ Les variables définies à l'intérieur d'une fonction ont une portée locale à cette fonction
- Quel(s) test(s) sont vrais si et seulement si l'entier `n` est impair ?
☒ `n%2!=0` ☒ `n%2==1` ☐ `n//2==1` ☐ `2%n==1` ☒ `not(n%2==0)`
- Si `lst` est une liste (type `list`), cocher les instructions valides (qui ne provoquent pas d'erreur)
☒ `lst + [3]` ☐ `lst + 3` ☒ `lst * 2` ☒ `lst[0] = 1` ☐ `lst(0)`
- Parmi les types de Python suivants, lesquels sont *mutables* (modifiables en place) ?
☐ `int` ☐ `float` ☐ `str` ☒ `list` ☐ `tuple`
- On suppose que `x` est un entier valant 8, quelles expressions seront évaluées à `True` ?
☒ `x>5 and x<10` ☒ `not(x!=8)` ☐ `x%4==0 and x%3==0` ☒ `10>x==8`
- On suppose que `s = 0` a été exécuté. Parmi les boucles suivantes, lesquelles permettent de calculer dans `s` la somme des éléments de `lst` ?

☒

```
for elt in lst:
    s = s + elt
```

☒

```
for i in range(len(lst)):
    s = s + lst[i]
```

☐

```
for i in range(len(lst)):
    s = s + i
```

☐

```
for elt in lst:
    s = s + lst[elt]
```
- Quelles sont les affirmations vraies concernant le type `str` de Python ?
☐ On peut modifier directement un caractère d'une chaîne par affectation : `s[0] = 'a'`
☒ On peut concaténer deux chaînes de caractères à l'aide de l'opérateur `+`
☒ La fonction `len(s)` renvoie le nombre de caractères de la chaîne `s`
☐ Une chaîne de caractères ne peut pas être vide
☒ L'instruction `s = ""` permet de définir une chaîne vide
- Si `lst` est la liste `[10, 20, 30, 40, 50]`, quelles tranches permettent d'obtenir la liste `[20, 30]` ?
☒ `lst[1:3]` ☐ `lst[1:2]` ☒ `lst[-4:-2]` ☒ `lst[1:-2]`
- Après exécution de l'instruction `l = [3*i for i in range(1, 6)]`, quelles affirmations concernant `l` sont vraies ?
☒ `len(l)` vaut 5
☐ 0 est l'un des éléments de `l`
☒ 1 est la liste `[3, 6, 9, 12, 15]`
☒ Le dernier élément de `l` est 15

□ Exercice 3 : Définir une fonction

Écrire en Python une fonction `nb_pairs` qui prend en argument une liste d'entiers `lst` et renvoie le nombre d'éléments pairs contenus dans cette liste. Par exemple, `nb_pairs([2, 5, 8, 11, 14])` doit renvoyer 3, et `nb_pairs([1, 3, 7])` doit renvoyer 0.

```
1 def nb_pairs(lst):
2     nb = 0
3     for elt in lst:
4         if elt % 2 == 0:
5             nb += 1
6     return nb
```

□ Exercice 4 : Exercice bonus

Écrire une fonction `deuxmax` qui prend en argument une liste d'entiers contenant au moins deux éléments et qui renvoie les deux plus grands éléments de cette liste. Par exemple `deuxmax([14, -2, 6, 20, 8, 3])` renvoie 14, 20.

```
1 def deuxmax(lst):
2     assert len(lst) >= 2, "La liste doit avoir au moins deux éléments"
3     if lst[0] > lst[1]:
4         max1 = lst[0]
5         max2 = lst[1]
6     else:
7         max1 = lst[1]
8         max2 = lst[0]
9     for i in range(2, len(lst)):
10        if lst[i] > max1:
11            max2 = max1
12            max1 = lst[i]
13        elif lst[i] > max2:
14            max2 = lst[i]
15    return max2, max1
```